```
# ==========================================================
# 1. Libraries
# ==========================================================
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.metrics import (
    roc_auc_score, roc_curve,
    confusion_matrix, classification_report
)

from sklearn.linear_model import LogisticRegression, SGDClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

from imblearn.over_sampling import SMOTE
from imblearn.over_sampling import SMOTEN
```

```
# ==========================================================
# 2️⃣ LOAD DATA
# ==========================================================
data = pd.read_stata(r"C:\Users\hp\Documents\Materials\Machine Learning\data\imputeddata3.dta")

# Encode target variable (Yes/No -> 1/0)
data['wasting'] = data['wasting'].map({'No':0, 'Yes':1})

target = "wasting"   # <-- CHANGE if needed
y = data[target]
X = data.drop(columns=[target])

print("Data shape:", data.shape)
print("Target distribution:\n", data['wasting'].value_counts())
```

```
Data shape: (76224, 27)
Target distribution:
 wasting
0    71592
1     4632
Name: count, dtype: int64
```

```
# ==========================================================
# 3. Define Categorical & Numeric Variables
# ==========================================================
categorical_cols = [
    'matage', 'placeresid', 'latrine', 'famsize', 'num5child',
    'sexofhead', 'litracy', 'wealth', 'bottlefeeding',
    'disposalofchild', 'maritalstatus', 'working', 'twin',
    'childgender', 'childage', 'birthinterval', 'breastfed',
    'anc', 'birthsize', 'diarrhea', 'fever', 'vitamina',
    'maternaldecision', 'dietdivers', 'watersource', 'ari',
    'country'
]

numeric_cols = [c for c in X.columns if c not in categorical_cols]
```

```
# ==========================================================
# 4. Train / Test Split
# ==========================================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    stratify=y,
    random_state=42)
```

```
# ==========================================================
# 5️⃣ Encoding (KeyError-safe)
```

```
# ============================================================
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

categorical_cols = X.select_dtypes(include=['object', 'category']).columns.tolist()
numeric_cols = X.select_dtypes(exclude=['object', 'category']).columns.tolist()

preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
         categorical_cols),
        ('num', 'passthrough', numeric_cols)
    ],
    remainder='drop'
)

X_train_enc = preprocessor.fit_transform(X_train)
X_test_enc  = preprocessor.transform(X_test)

# ✅ DO NOT force column names (prevents KeyError)
X_train_enc = pd.DataFrame(X_train_enc)
X_test_enc  = pd.DataFrame(X_test_enc)

print("Encoded shapes:")
print("Train:", X_train_enc.shape)
print("Test :", X_test_enc.shape)
```

```
Encoded shapes:
Train: (60979, 78)
Test : (15245, 78)
```

```
# ============================================================
# 7️⃣ APPLY SMOTEN TO TRAINING DATA
# ============================================================
smote = SMOTEN(random_state=42)
X_train_bal, y_train_bal = smote.fit_resample(X_train_enc, y_train)

print("\nAfter SMOTEN, training class distribution:")
print(pd.Series(y_train_bal).value_counts())
```

```
After SMOTEN, training class distribution:
wasting
0    57273
1    57273
Name: count, dtype: int64
```

```
from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier
```

```
scale_pos = len(y_train_bal[y_train_bal == 0]) / len(y_train_bal[y_train_bal == 1])

models = {
    # ===== Classical =====
    "Logistic Regression": LogisticRegression(
        max_iter=1000,
        solver='liblinear'),

    "Random Forest": RandomForestClassifier(
        n_estimators=500,
        random_state=42,n_jobs=-1),

    "Decision Tree": DecisionTreeClassifier(max_depth=10, random_state=42),

    "KNN": KNeighborsClassifier(
        n_neighbors=15
    ),

    "Naive Bayes": GaussianNB(),
```

```
        "SVM": SVC(
            probability=True,
            class_weight='balanced',
            random_state=42
        ),

        # ===== Boosting / Linear Extensions =====
        "XGBoost": XGBClassifier(
            scale_pos_weight=scale_pos,
            eval_metric='logloss',
            random_state=42,
            verbosity=0
        ),

        "LightGBM": LGBMClassifier(
            class_weight='balanced',
            random_state=42,
            verbose=-1
        ),

        "CatBoost": CatBoostClassifier(
            class_weights=[1, scale_pos],
            random_state=42,
            verbose=0
        ),

        "SGD": SGDClassifier(
            loss='log_loss',
            class_weight='balanced',
            random_state=42
        )
    }
```

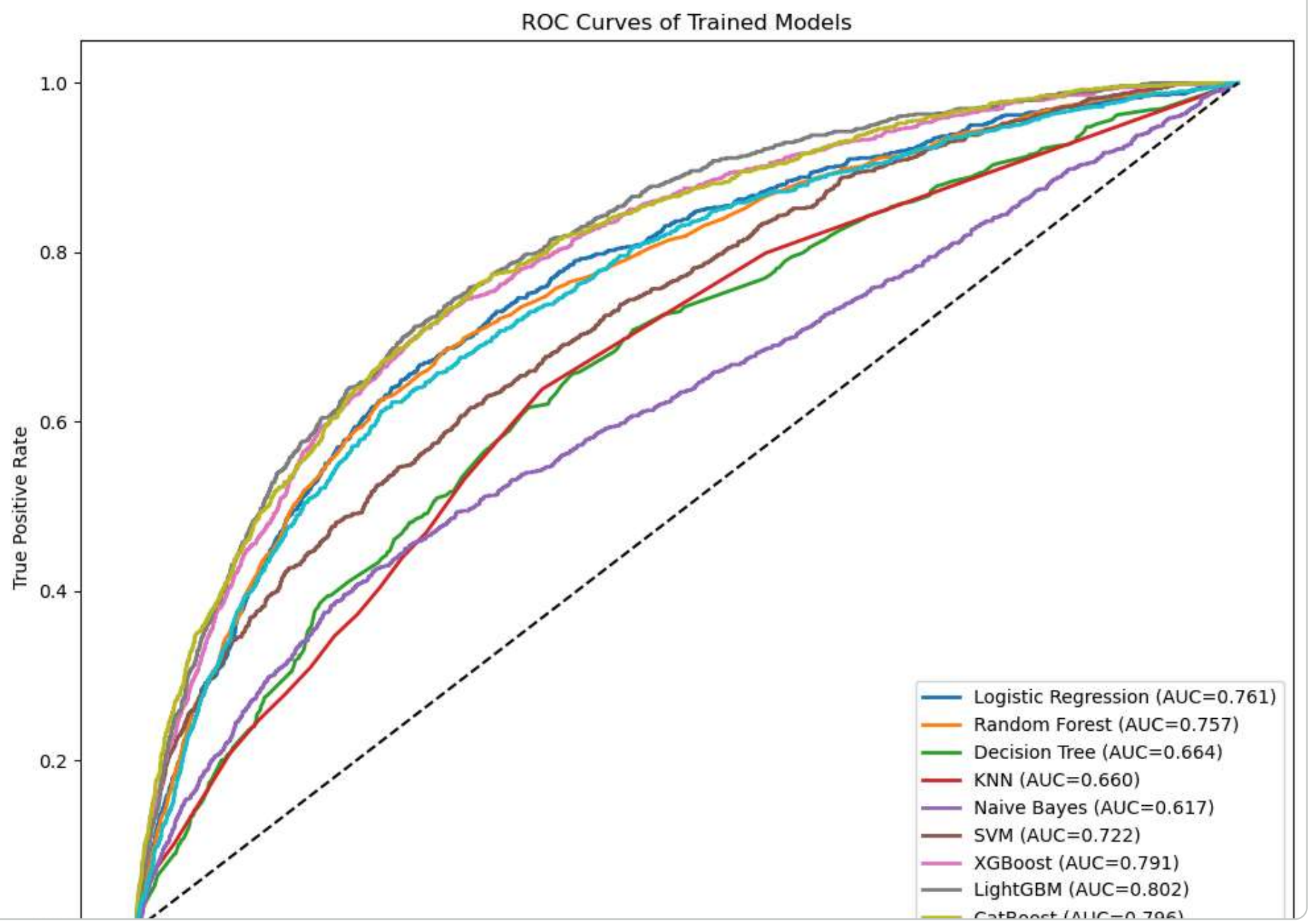
```
    from sklearn.metrics import (
        roc_auc_score, roc_curve,
        confusion_matrix, classification_report
    )

    results = []

    for name, model in models.items():

        print("\n" + "="*65)
        print(f"Training: {name}")

        # Train
        model.fit(X_train_bal, y_train_bal)

        # ----- Probability handling (IMPORTANT) -----
        if hasattr(model, "predict_proba"):
            y_prob = model.predict_proba(X_test_enc)[:, 1]
        else:
            # For models like SGD
            scores = model.decision_function(X_test_enc)
            y_prob = (scores - scores.min()) / (scores.max() - scores.min())

        # AUC
        auc = roc_auc_score(y_test, y_prob)

        # Youden Index
        fpr, tpr, thresholds = roc_curve(y_test, y_prob)
        youden = tpr - fpr
        best_idx = np.argmax(youden)
        best_thresh = thresholds[best_idx]

        # Classification
        y_pred = (y_prob >= best_thresh).astype(int)

        # Output
        print(f"AUC: {auc:.4f}")
        print(f"Optimal Threshold (Youden): {best_thresh:.4f}")
        print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
        print("Classification Report:\n",
              classification_report(y_test, y_pred))
```

```
    results.append([name, auc, best_thresh])
```

```
==================================================================
Training: Logistic Regression
AUC: 0.7613
Optimal Threshold (Youden): 0.0846
Confusion Matrix:
 [[10633  3686]
 [  308   618]]
Classification Report:
               precision    recall  f1-score   support

           0       0.97      0.74      0.84     14319
           1       0.14      0.67      0.24       926

    accuracy                           0.74     15245
   macro avg       0.56      0.70      0.54     15245
weighted avg       0.92      0.74      0.81     15245


==================================================================
Training: Random Forest
AUC: 0.7568
Optimal Threshold (Youden): 0.0920
Confusion Matrix:
 [[10318  4001]
 [  293   633]]
Classification Report:
               precision    recall  f1-score   support

           0       0.97      0.72      0.83     14319
           1       0.14      0.68      0.23       926

    accuracy                           0.72     15245
   macro avg       0.55      0.70      0.53     15245
weighted avg       0.92      0.72      0.79     15245


==================================================================
Training: Decision Tree
AUC: 0.6637
Optimal Threshold (Youden): 0.0642
Confusion Matrix:
 [[9220 5099]
 [ 355  571]]
Classification Report:
               precision    recall  f1-score   support

           0       0.96      0.64      0.77     14319
           1       0.10      0.62      0.17       926

    accuracy                           0.64     15245
   macro avg       0.53      0.63      0.47     15245
weighted avg       0.91      0.64      0.74     15245


==================================================================
Training: KNN
AUC: 0.6601
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,8))

for name, model in models.items():

    # DO NOT FIT AGAIN

    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(X_test_enc)[:, 1]
    else:
        scores = model.decision_function(X_test_enc)
        y_prob = (scores - scores.min()) / (scores.max() - scores.min())

    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc = roc_auc_score(y_test, y_prob)

    plt.plot(fpr, tpr, linewidth=2, label=f"{name} (AUC={auc:.3f})")

plt.plot([0,1],[0,1],'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
```

```
plt.title("ROC Curves of Trained Models")
plt.legend()
plt.tight_layout()
plt.show()
```



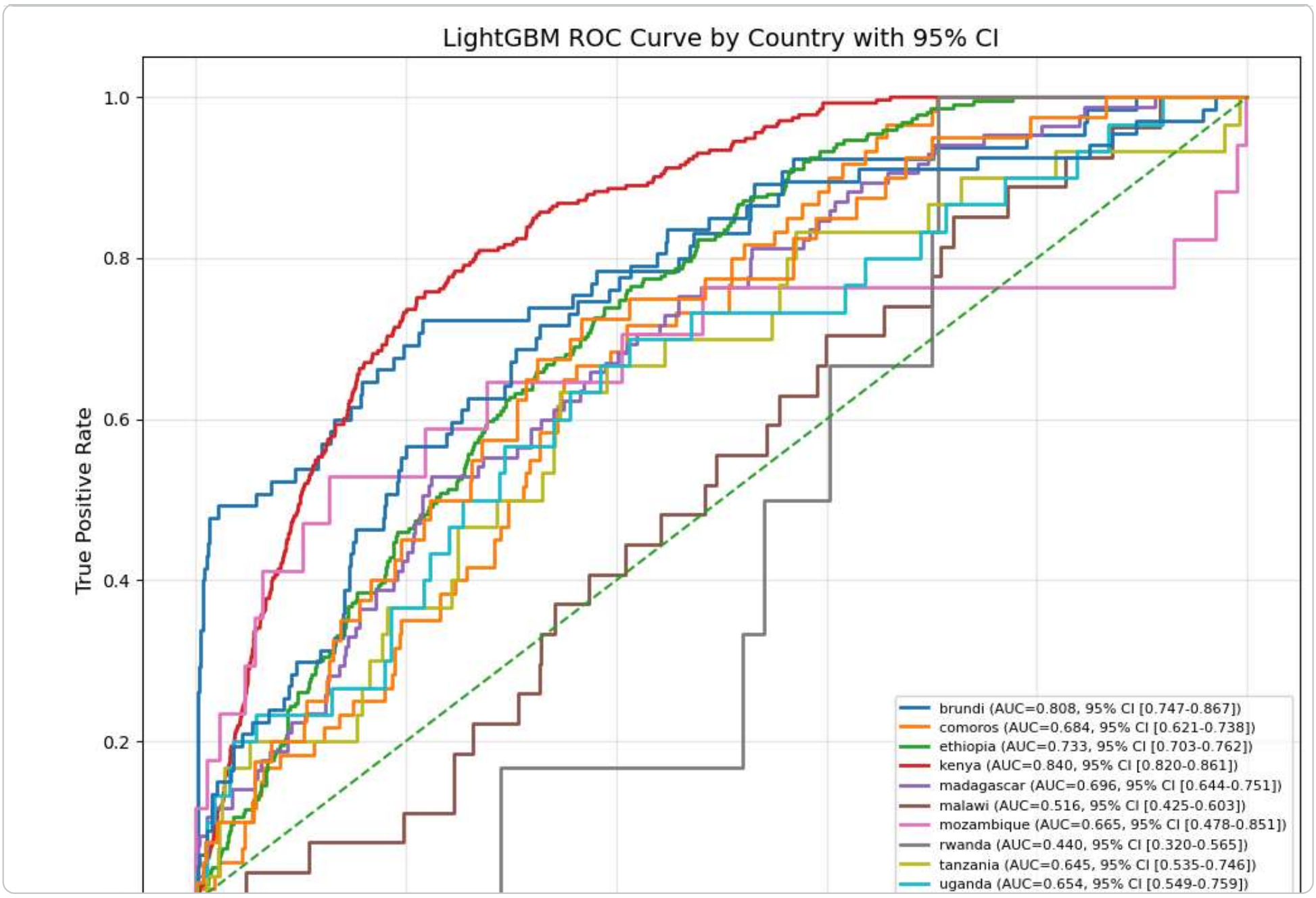

```
# ---------------------------------------------------------
# 1️⃣ Get already trained LightGBM model
# ---------------------------------------------------------
lgbm_model = models["LightGBM"]   # already fitted earlier


# ---------------------------------------------------------
# 2️⃣ Save model (optional)
# ---------------------------------------------------------
import joblib
joblib.dump(lgbm_model, "lightgbm_final_model.pkl")


# ---------------------------------------------------------
# 3️⃣ Prepare test dataframe with country info
# ---------------------------------------------------------
country_test = X_test["country"].values

test_df = pd.DataFrame(X_test_enc)
test_df["true"] = y_test.values
test_df["country"] = country_test
best_thresh = 0.0777

# ---------------------------------------------------------
# 4️⃣ Country-wise Performance
# ---------------------------------------------------------
from sklearn.metrics import (
    roc_auc_score,
    accuracy_score,
    recall_score,
    precision_score,
    f1_score
)

results = []

for c in np.unique(country_test):
```

```
    sub = test_df[test_df["country"] == c]

    if sub["true"].nunique() < 2:
        continue

    X_c = sub.drop(columns=["true", "country"])
    y_c = sub["true"]

    probs = lgbm_model.predict_proba(X_c)[:, 1]
    preds = (probs >= best_thresh).astype(int)

    auc  = roc_auc_score(y_c, probs)
    acc  = accuracy_score(y_c, preds)
    sens = recall_score(y_c, preds)
    prec = precision_score(y_c, preds)
    f1   = f1_score(y_c, preds)

    results.append([c, len(sub), auc, acc, sens, prec, f1])

country_results = pd.DataFrame(
    results,
    columns=["Country","SampleSize","AUC","Accuracy",
             "Sensitivity","Precision","F1"]
).sort_values("AUC", ascending=False)

country_results
```

	Country	SampleSize	AUC	Accuracy	Sensitivity	Precision	F1
3	kenya	3527	0.840207	0.671676	0.868132	0.174393	0.290441
0	brundi	1252	0.808178	0.862620	0.569231	0.204420	0.300813
10	zambia	1708	0.735245	0.823185	0.462687	0.104377	0.170330
2	ethiopia	1788	0.732967	0.460291	0.938053	0.182287	0.305256
11	zimbabwe	958	0.703513	0.865344	0.250000	0.091743	0.134228
4	madagascar	1167	0.696107	0.613539	0.670588	0.118750	0.201770
1	comoros	494	0.683602	0.457490	0.900000	0.170886	0.287234
6	mozambique	764	0.664698	0.904450	0.411765	0.100000	0.160920
9	uganda	904	0.654233	0.860619	0.233333	0.063636	0.100000
8	tanzania	939	0.644591	0.869010	0.200000	0.057143	0.088889
5	malawi	979	0.515795	0.886619	0.037037	0.011628	0.017699
7	rwanda	765	0.440053	0.979085	0.000000	0.000000	0.000000

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.utils import resample

# ---------------------------------------------------
# 1️⃣ Ensure best threshold is defined (from earlier)
# ---------------------------------------------------
best_thresh = 0.0777   # <-- replace if your final LightGBM threshold differs

# ---------------------------------------------------
# 2️⃣ Bootstrap AUC CI function
# ---------------------------------------------------
def bootstrap_auc_ci(y_true, y_prob, n_bootstrap=1000, seed=42):
    rng = np.random.RandomState(seed)
    bootstrapped_scores = []

    for i in range(n_bootstrap):
        indices = rng.randint(0, len(y_prob), len(y_prob))

        if len(np.unique(y_true.iloc[indices])) < 2:
            continue

        score = roc_auc_score(y_true.iloc[indices],
                              y_prob[indices])
        bootstrapped_scores.append(score)
```

```
    sorted_scores = np.sort(bootstrapped_scores)

    lower = sorted_scores[int(0.025 * len(sorted_scores))]
    upper = sorted_scores[int(0.975 * len(sorted_scores))]

    return lower, upper

# ---------------------------------------------------
# 3️⃣ Plot ROC Curve for Each Country
# ---------------------------------------------------
plt.figure(figsize=(10, 8))

for c in np.unique(country_test):

    sub = test_df[test_df["country"] == c]

    if sub["true"].nunique() < 2:
        continue

    X_c = sub.drop(columns=["true", "country"])
    y_c = sub["true"]

    probs = lgbm_model.predict_proba(X_c)[:, 1]

    # ROC
    fpr, tpr, _ = roc_curve(y_c, probs)
    auc = roc_auc_score(y_c, probs)

    # 95% CI
    lower_ci, upper_ci = bootstrap_auc_ci(y_c.reset_index(drop=True),
                                          probs)

    plt.plot(fpr, tpr, linewidth=2,
             label=f"{c} (AUC={auc:.3f}, 95% CI [{lower_ci:.3f}-{upper_ci:.3f}])")

# Random line
plt.plot([0, 1], [0, 1], linestyle='--')

plt.xlabel("False Positive Rate", fontsize=12)
plt.ylabel("True Positive Rate", fontsize=12)
plt.title("LightGBM ROC Curve by Country with 95% CI", fontsize=14)
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

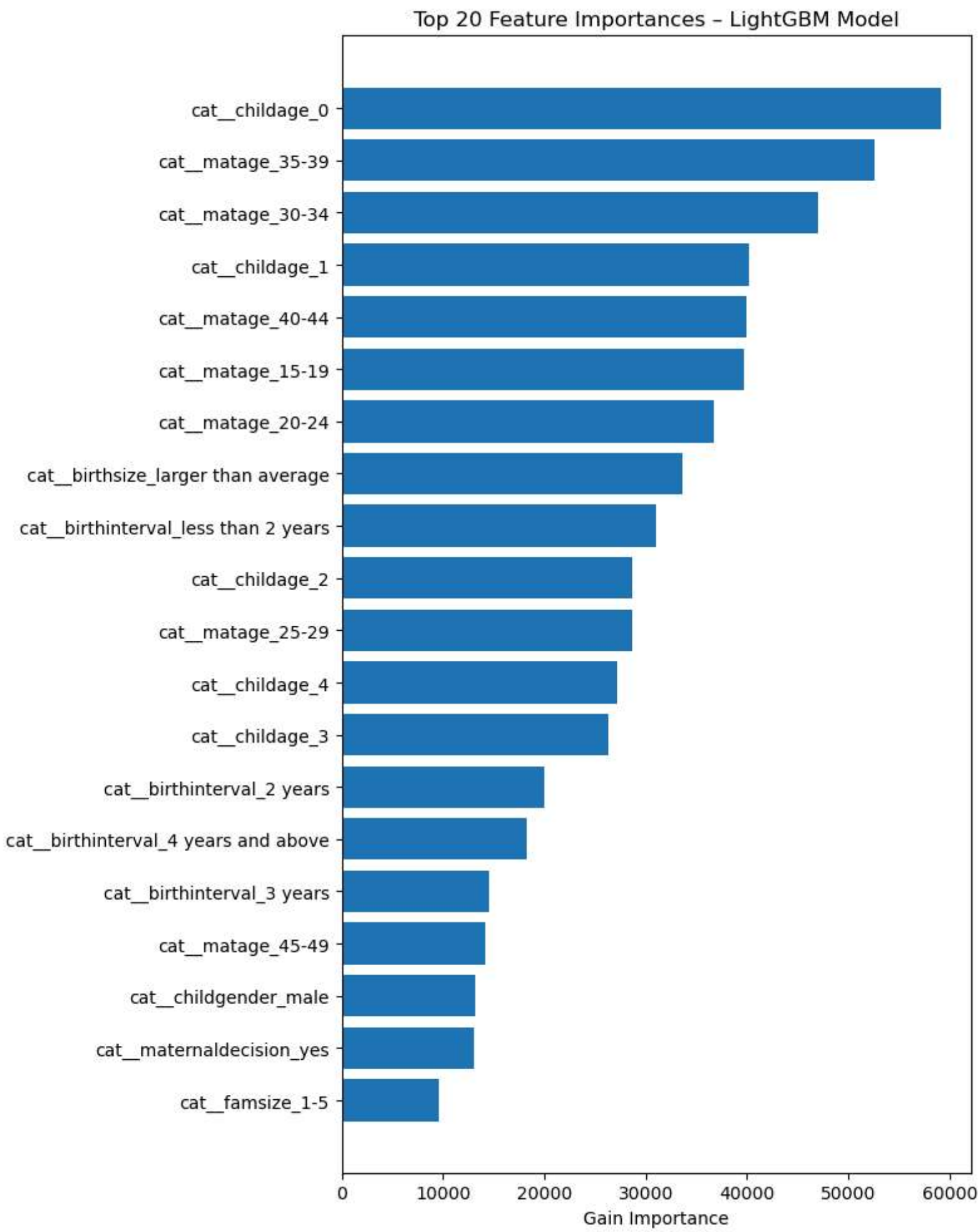



```
# 1 Extract Feature Importance from Your Final LightGBM Model

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# ---------------------------------------------------
# Get feature names
# ---------------------------------------------------
feature_names = preprocessor.get_feature_names_out()

# ---------------------------------------------------
# Get importance (GAIN is recommended)
# ---------------------------------------------------
importance = lgbm_model.booster_.feature_importance(importance_type='gain')

# Create dataframe
feat_imp = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importance
})

# Sort
feat_imp = feat_imp.sort_values("Importance", ascending=False)

# Display top 20
print(feat_imp.head(20))
```

```
                                Feature    Importance
32                      cat__childage_0  59122.441045
4                     cat__matage_35-39  52602.636260
3                     cat__matage_30-34  47002.750877
33                      cat__childage_1  40250.202321
5                     cat__matage_40-44  40002.924261
0                     cat__matage_15-19  39755.250086
1                     cat__matage_20-24  36744.315500
48    cat__birthsize_larger than average  33674.991913
40  cat__birthinterval_less than 2 years  31083.959104
34                      cat__childage_2  28724.937990
2                     cat__matage_25-29  28644.438557
36                      cat__childage_4  27134.309412
35                      cat__childage_3  26331.950919
37            cat__birthinterval_2 years  19948.385891
39  cat__birthinterval_4 years and above  18313.868531
```

```
38            cat__birthinterval_3 years  14522.438146
6                     cat__matage_45-49  14144.720634
31                cat__childgender_male  13173.060419
59             cat__maternaldecision_yes  13068.885932
11                     cat__famsize_1-5   9541.806360
```

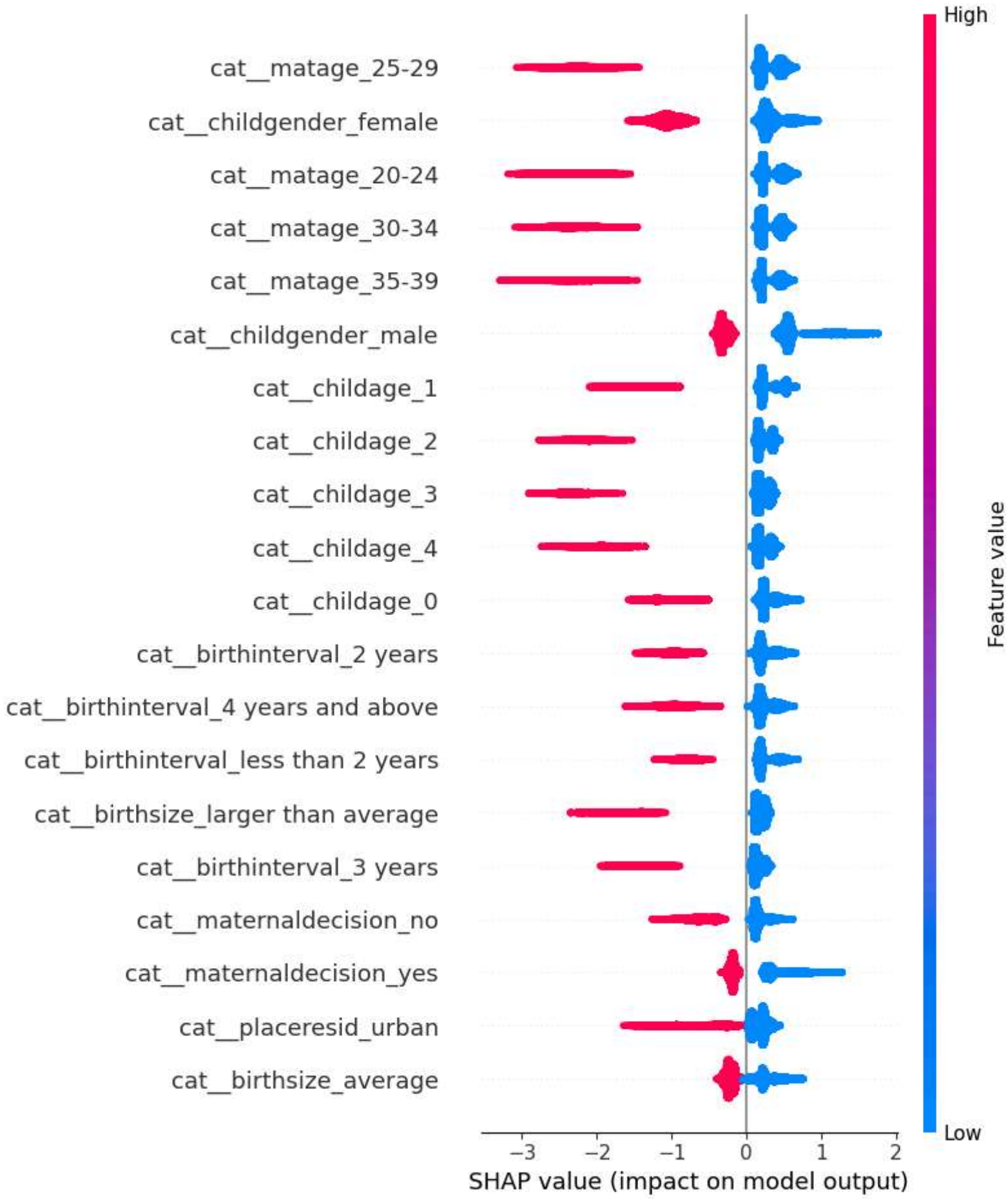
```
top_n = 20
top_features = feat_imp.head(top_n)

plt.figure(figsize=(8, 10))
plt.barh(top_features["Feature"][::-1],
         top_features["Importance"][::-1])

plt.xlabel("Gain Importance")
plt.title("Top 20 Feature Importances – LightGBM Model")
plt.tight_layout()
plt.show()
```




```
import shap

explainer = shap.TreeExplainer(lgbm_model)
shap_values = explainer.shap_values(X_train_bal)

shap.summary_plot(shap_values, X_train_bal,
```

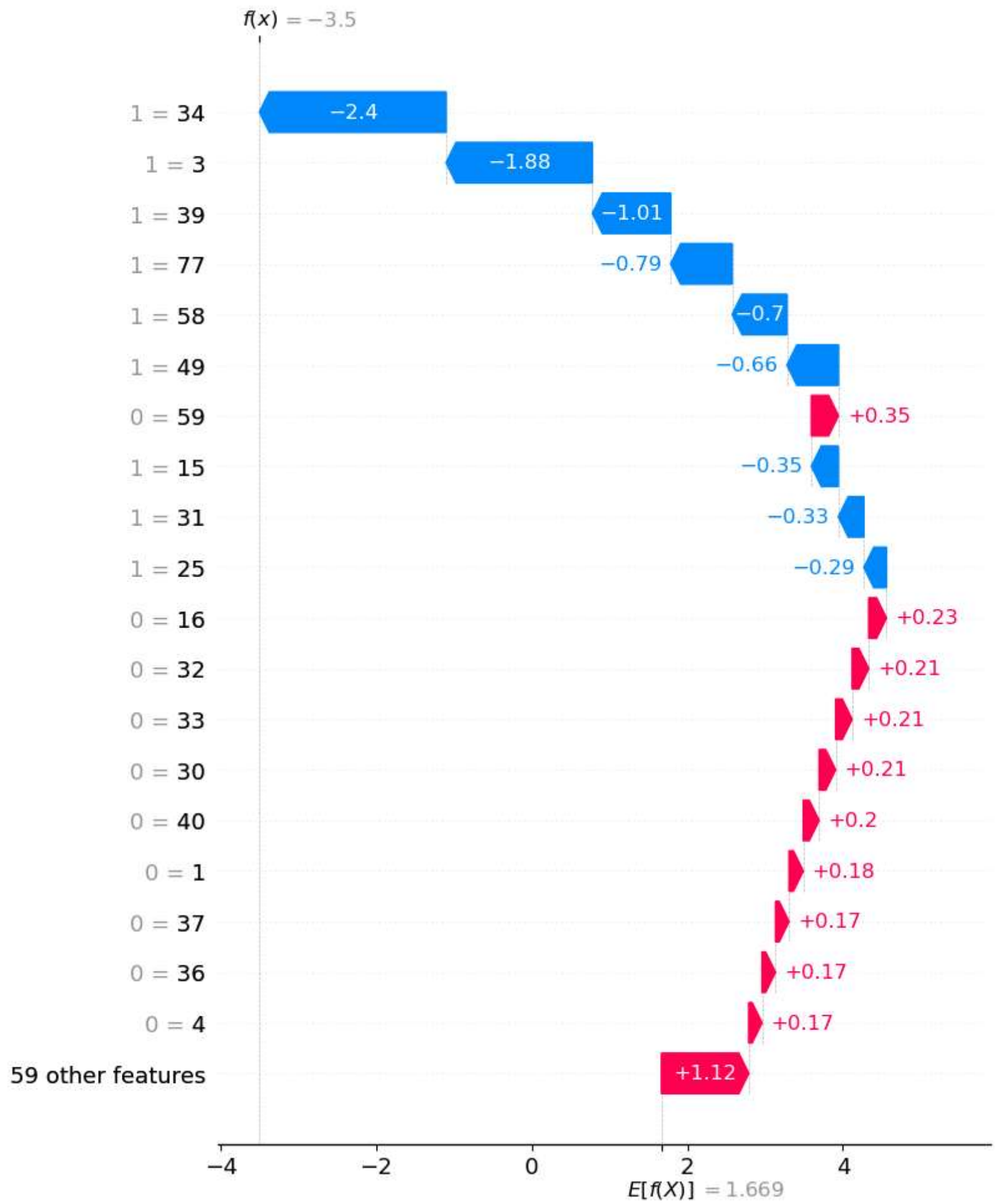
```
                    feature_names=feature_names)
```

```
C:\Users\hp\anaconda3\Lib\site-packages\shap\explainers\_tree.py:587: UserWarning: LightGBM binary classifier with Tre
  warnings.warn(
```

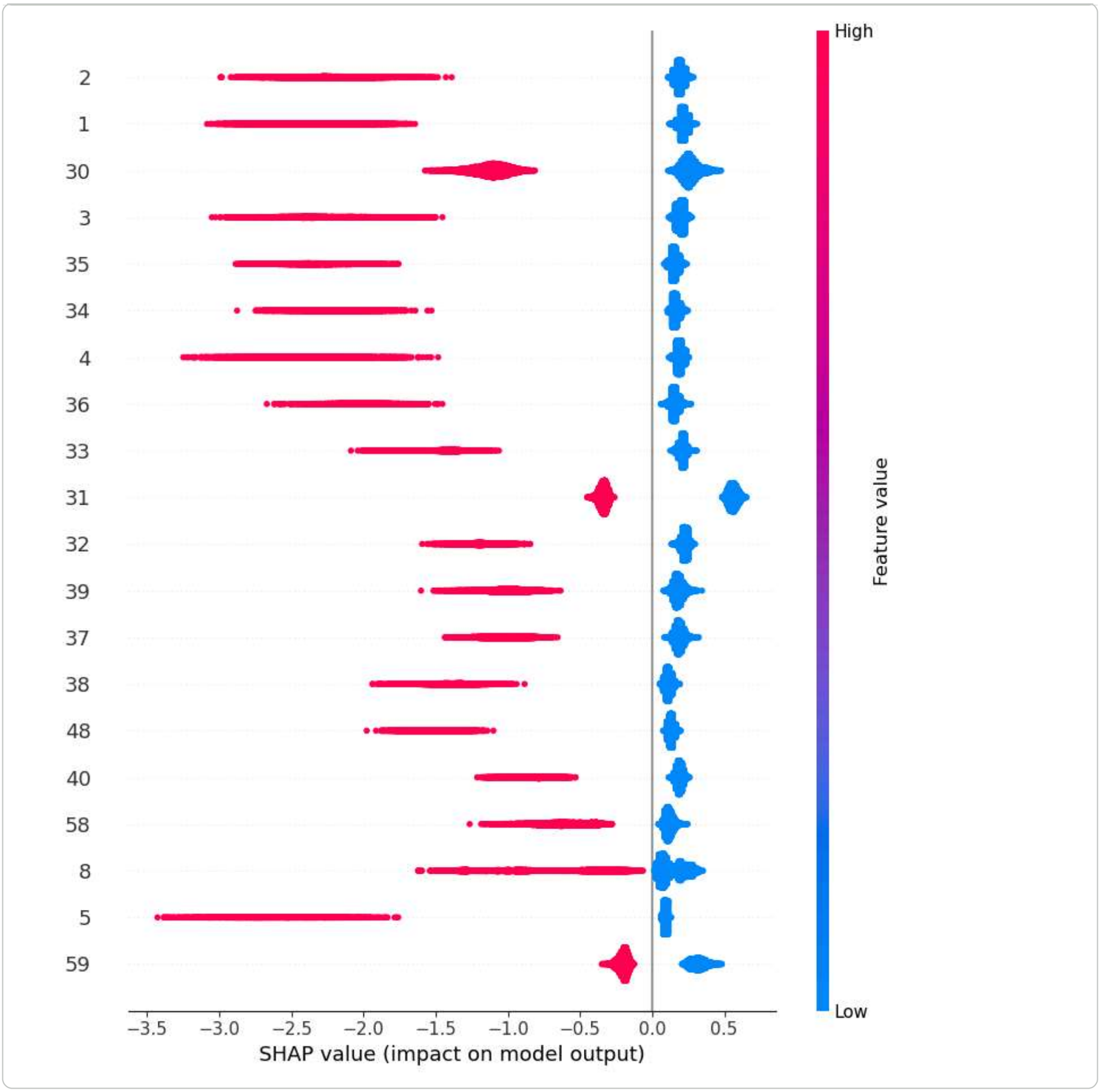



```
# ✅ Step 1 — Prepare SHAP Explainer
import shap

# Create explainer
explainer = shap.TreeExplainer(lgbm_model)

# Directly compute explanation object
shap_values = explainer(X_test_enc)
```

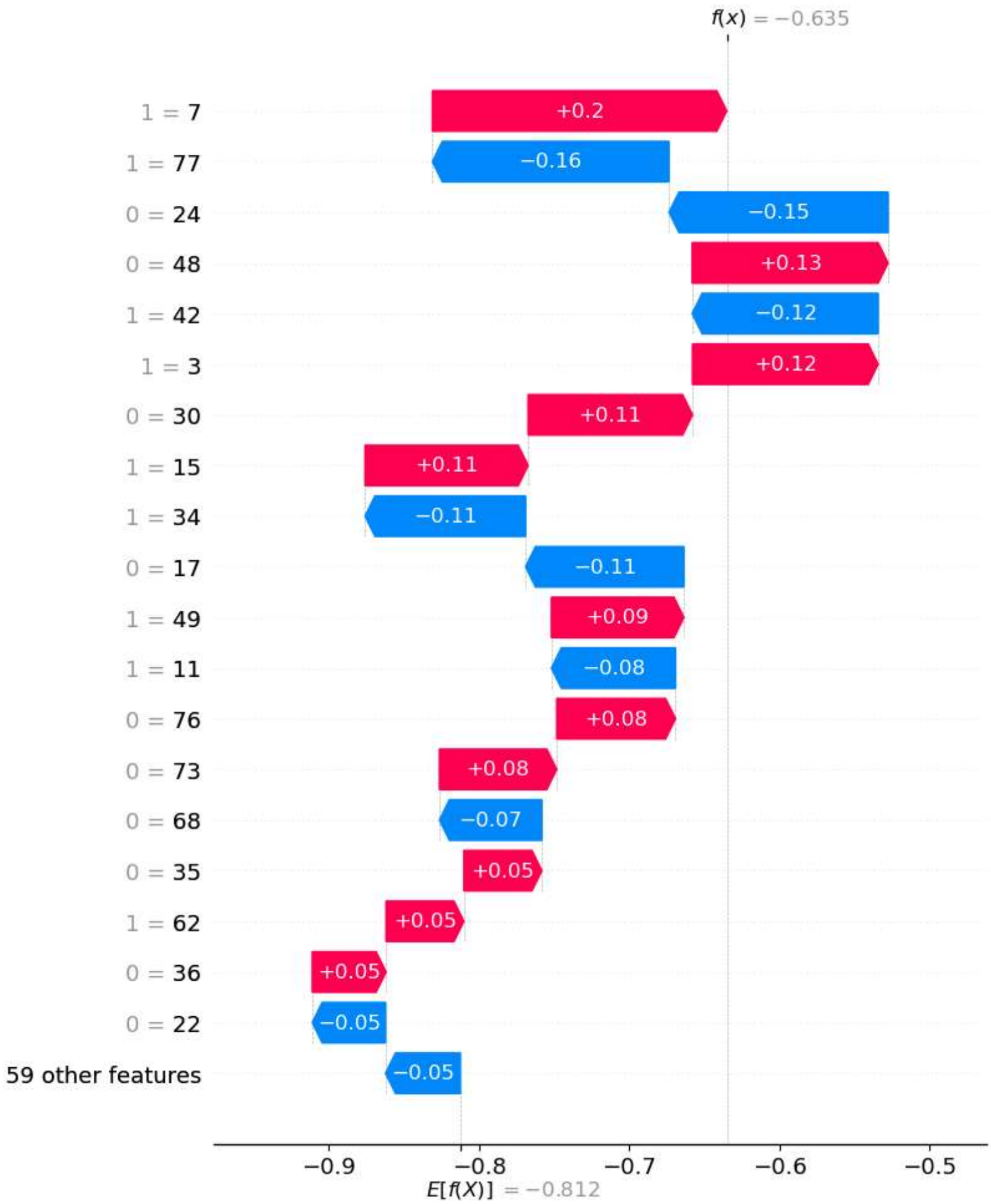
```
row_id = 0  # change to any index you want

shap.plots.waterfall(shap_values[row_id], max_display=20)
```




```
# ✅ Step 2 — Plot SHAP Beeswarm
import matplotlib.pyplot as plt

# Create the beeswarm plot
shap.summary_plot(shap_values, X_test_enc, plot_type="dot")

# Optional: if running in a script and the plot doesn't show
plt.show()
```




```
import shap
import pandas as pd


# 1 Ensure test data is a DataFrame
X_test_df = pd.DataFrame(X_test_enc, columns=X_train_enc.columns)

# 2 Create TreeExplainer
explainer = shap.TreeExplainer(models["LightGBM"])

# 3 Compute SHAP Explanation object
shap_expl = explainer(X_test_df)  # shap.Explanation object

# 4 Pick a row to plot
row_id = 0

# 5 For binary classifier, select positive class directly
# If shap_expl.values is 2D (n_samples x n_features), just pick the row
shap_row = shap.Explanation(
    values=shap_expl.values[row_id],       # 1D array of SHAP values
    base_values=shap_expl.base_values[row_id],  # scalar base value
    data=X_test_df.iloc[row_id],           # row data
    feature_names=X_test_df.columns
)

# 6 Plot waterfall
shap.plots.waterfall(shap_row, max_display=20)
```




```
import pandas as pd

# Make sure X_test_enc is a DataFrame with exact same column names
X_test_df = pd.DataFrame(X_test_enc, columns=X_train_enc.columns)
```

```
# Plot waterfall for a specific observation
row_id = 1256  # change index for other rows
shap.plots.waterfall(shap_values[row_id], max_display=20)
```

$f(x) = -5.366$